```
import matplotlib as mpl
import matplotlib.font_manager as fm
import os

# ── Registrar Arial (detecta automaticamente no sistema) ──────────────────
def _registrar_arial():
    fontes = [f.name for f in fm.fontManager.ttflist]
    if "Arial" in fontes:
        return

    candidatos = [
        r"C:\Windows\Fonts\arial.ttf",
        "/Library/Fonts/Arial.ttf",
        "/usr/share/fonts/truetype/msttcorefonts/Arial.ttf",
        os.path.join(os.path.dirname(__file__), "Arial.ttf"),
    ]
    for caminho in candidatos:
        if os.path.exists(caminho):
            fm.fontManager.addfont(caminho)
            return

    import urllib.request
    dest = os.path.join(os.path.expanduser("~"), "Arial.ttf")
    if not os.path.exists(dest):
        url = "https://github.com/matomo-org/travis-scripts/raw/master/fonts/Arial.ttf"
        urllib.request.urlretrieve(url, dest)
    fm.fontManager.addfont(dest)

_registrar_arial()
mpl.rcParams["font.family"] = "Arial"

from docx import Document
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.lines import Line2D
import numpy as np
import re

# ── Ajuste aqui se necessário ─────────────────────────────────────────────
DOCX_PATH  = "Sociodemograficos_ingles.docx"
OUTPUT_PNG = "forest_sociodemograficos.png"
# ──────────────────────────────────────────────────────────────────────────


#
# ═══════════════════════════════════════════════════════════════════════════
# 1. LEITURA DO DOCX
#
# ═══════════════════════════════════════════════════════════════════════════

def read_word_table(file_path, table_index=0):
    doc = Document(file_path)
    table = doc.tables[table_index]
    data = []
    for row in table.rows[1:]:
        data.append([cell.text.strip() for cell in row.cells])
    columns = [cell.text.strip() for cell in table.rows[0].cells]
```

```
    return pd.DataFrame(data, columns=columns)


def parse_or_ci(text):
    text = text.replace(",", ".").strip()
    match = re.search(r"([\d.]+)\s*\(([\d.]+)\s*[-–]\s*([\d.]+)\)", text)
    if match:
        try:
            or_ = float(match.group(1))
            lo  = float(match.group(2))
            hi  = float(match.group(3))
            if or_ <= 0 or lo <= 0 or hi == float("inf") or or_ < 1e-6:
                return None, None, None
            return or_, lo, hi
        except ValueError:
            return None, None, None
    return None, None, None


def parse_p(text):
    text = text.replace(",", ".").strip()
    if text.startswith("<"):
        try:
            return float(text[1:]) / 2
        except ValueError:
            return None
    try:
        return float(text)
    except ValueError:
        return None


# ── Carregar e limpar ──────────────────────────────────────────────────────
df_raw = read_word_table(DOCX_PATH)
df_raw.columns = ["label", "n_pct", "OR_IC", "p_OR", "ORa_IC", "p_ORa"]
df_raw["label"] = df_raw["label"].str.replace("\n", " / ", regex=False)

df_raw[["OR",  "IC_inf",  "IC_sup"]]  = df_raw["OR_IC"].apply(
    lambda x: pd.Series(parse_or_ci(x)))
df_raw[["ORa", "ICa_inf", "ICa_sup"]] = df_raw["ORa_IC"].apply(
    lambda x: pd.Series(parse_or_ci(x)))

df_raw["p_OR"]  = df_raw["p_OR"].apply(parse_p)
df_raw["p_ORa"] = df_raw["p_ORa"].apply(parse_p)

df_raw["is_ref"]    = df_raw["OR_IC"].str.strip().str.lower() == "ref."
df_raw["is_header"] = (df_raw["n_pct"].str.strip() == "") & (~df_raw["is_ref"])

print("=" * 70)
print(f"Tabela lida de: {DOCX_PATH}")
print(df_raw[["label", "n_pct", "OR", "IC_inf", "IC_sup",
              "ORa", "ICa_inf", "ICa_sup", "is_ref", "is_header"]].to_string())
print("=" * 70)


#
# ═══════════════════════════════════════════════════════════════════════════
# 2. FOREST PLOT
```

```
#
═══════════════════════════════════════════════════════════════════════════════

BG        = "#FFFFFF"
PANEL     = "#F6F8FA"
BORDER    = "#D0D7DE"
TEXT      = "#1F2328"
SUBTEXT   = "#57606A"
GOLD      = "#B08800"
COR_PROT  = "#1D9E75"
COR_RISCO = "#E07B39"


def sig_stars(p):
    if p is None:  return ""
    if p < 0.001:  return "***"
    if p < 0.01:   return "**"
    if p < 0.05:   return "*"
    return ""


n_rows = len(df_raw)
fig_h  = max(10, n_rows * 0.56 + 3.0)

fig, axes = plt.subplots(
    1, 4, figsize=(20, fig_h), facecolor=BG,
    gridspec_kw={"width_ratios": [2.2, 3, 1.2, 3], "wspace": 0.10}
)
ax_labels, ax_or, ax_gap, ax_ora = axes

ax_gap.set_visible(False)

# ── Painel de labels
──────────────────────────────────────────────────────────────
ax_labels.set_facecolor(BG)
ax_labels.set_xlim(0, 1)
ax_labels.set_ylim(n_rows - 0.5, -0.5)
for spine in ax_labels.spines.values():
    spine.set_visible(False)
ax_labels.set_xticks([])
ax_labels.set_yticks([])

# Faixas zebradas
for ax in (ax_labels, ax_or, ax_ora):
    stripe = 0
    for i, row in df_raw.iterrows():
        if not row["is_header"]:
            bg_col = "#E8EDF2" if stripe % 2 == 0 else PANEL
            ax.axhspan(i - 0.42, i + 0.42, color=bg_col, alpha=0.55, zorder=0)
            stripe += 1

# Labels (fonte Arial explícita em cada ax.text)
for i, row in df_raw.iterrows():
    lbl = row["label"]
    if row["is_header"]:
        lbl = lbl.split("/")[0].strip()
        ax_labels.text(0.98, i, lbl, color=TEXT,
                       fontsize=25, fontweight="bold",
                       va="center", ha="right", fontfamily="Arial")
    elif row["is_ref"]:
```

```
        ax_labels.text(0.95, i, f"   {lbl}", color=SUBTEXT,
                       fontsize=20, va="center", ha="right", fontfamily="Arial")
    else:
        ax_labels.text(0.95, i, f"   {lbl}", color=TEXT,
                       fontsize=20, va="center", ha="right", fontfamily="Arial")


# ── Função genérica de painel OR ──────────────────────────────────────────
def draw_panel(ax, col_or, col_lo, col_hi, col_p, title, xlim):
    ax.set_facecolor(PANEL)
    ax.set_xscale("log")
    ax.xaxis.grid(True, color=BORDER, linewidth=1.7, zorder=0, alpha=0.8)
    ax.set_axisbelow(True)
    for spine in ax.spines.values():
        spine.set_edgecolor(BORDER)
        spine.set_linewidth(0.8)

    ax.axvline(1.0, color=GOLD, linewidth=2.3, linestyle="--", zorder=2, alpha=0.9)

    for i, row in df_raw.iterrows():
        if row["is_header"]:
            continue

        if row["is_ref"]:
            ax.text(0.6, i, "ref.", color=SUBTEXT, fontsize=18,
                    va="center", ha="center", fontfamily="Arial",
                    transform=ax.get_yaxis_transform())
            continue

        OR = row[col_or]
        lo = row[col_lo]
        hi = row[col_hi]
        p  = row[col_p]

        if pd.isna(OR) or pd.isna(lo) or pd.isna(hi):
            ax.text(0.5, i, "—", color=SUBTEXT, fontsize=15,
                    va="center", ha="center", fontfamily="Arial",
                    transform=ax.get_yaxis_transform())
            continue

        cor = COR_PROT if OR < 1 else COR_RISCO
        sig = (p is not None) and not np.isnan(p) and (p < 0.05)

        lo_plot = max(lo, xlim[0] * 1.02)
        hi_plot = min(hi, xlim[1] * 0.98)
        ax.plot([lo_plot, hi_plot], [i, i],
                color=cor, linewidth=3.8, zorder=3, alpha=0.85,
                solid_capstyle="round")

        ax.plot(OR, i,
                marker="D" if sig else "o",
                markersize=8 if sig else 7,
                color=cor,
                markerfacecolor=cor if sig else BG,
                markeredgecolor=cor,
                markeredgewidth=1.6,
                zorder=5)

        hi_txt = min(hi, 9999)
```

```
        txt = f"{OR:.2f} ({lo:.2f}–{hi_txt:.2f}){sig_stars(p)}"
        ax.text(1.02, i, txt,
                transform=ax.get_yaxis_transform(),
                color=TEXT, fontsize=21, va="center", ha="left",
                fontfamily="Arial", clip_on=False)

    ax.set_yticks(range(n_rows))
    ax.set_yticklabels([""] * n_rows)
    ax.tick_params(axis="y", length=0)
    ax.tick_params(axis="x", colors=SUBTEXT, labelsize=12)

    # tick labels do eixo X — tick_params não aceita fontfamily
    for lbl in ax.get_xticklabels():
        lbl.set_fontfamily("Arial")

    ax.set_ylim(n_rows - 0.5, -0.5)
    ax.set_xlim(*xlim)
    ax.set_xlabel("Odds Ratio (scale log)", fontsize=22,
                  color=SUBTEXT, labelpad=8, fontfamily="Arial")
    ax.set_title(title, fontsize=25, fontweight="bold",
                 color=TEXT, pad=12, fontfamily="Arial")


draw_panel(ax_or,  "OR",  "IC_inf",  "IC_sup",  "p_OR",
           "OR bruto (IC 95%)",    xlim=(0.05, 30))
draw_panel(ax_ora, "ORa", "ICa_inf", "ICa_sup", "p_ORa",
           "OR ajustado (IC 95%)", xlim=(0.05, 800))


# ── Legenda ──────────────────────────────────────────────────────────────────
legend_elements = [
    mpatches.Patch(facecolor=COR_PROT,  edgecolor=COR_PROT,
                   label="Protective factor (OR < 1)"),
    mpatches.Patch(facecolor=COR_RISCO, edgecolor=COR_RISCO,
                   label="Risk factor (OR > 1)"),
    Line2D([0], [0], marker="D", color="none",
           markerfacecolor=TEXT, markeredgecolor=TEXT,
           markersize=9, label="Diamond = p < 0,05"),
    Line2D([0], [0], marker="o", color="none",
           markerfacecolor=BG, markeredgecolor=TEXT,
           markeredgewidth=2.4, markersize=12,
           label="Open circle = p ≥ 0,05"),
    Line2D([0], [0], color=GOLD, linewidth=2.5,
           linestyle="--", label="Reference line (OR = 1)"),
]
ax_or.legend(handles=legend_elements, frameon=True,
             edgecolor=BORDER, facecolor=BG, labelcolor=TEXT,
             loc="lower right", framealpha=0.97,
             borderpad=0.9, handlelength=0.5,
             prop={"family": "Arial", "size": 10})

# ── Títulos e rodapé (fig.text) ──────────────────────────────────────────────
fig.text(0.5, 1.025,
         "Forest Plot — Crude and Adjusted Odds Ratio",
         ha="center", va="top",
         fontsize=30, fontweight="bold", color=TEXT, fontfamily="Arial")

fig.text(0.5, 0.989,
```

```
        "Sociodemographic variables  |  n = 703",
        ha="center", va="top",
        fontsize=25, color=SUBTEXT, fontfamily="Arial")

fig.add_artist(plt.Line2D(
   [0.03, 0.97], [0.958, 0.958],
   transform=fig.transFigure, color=BORDER, linewidth=1.8))

fig.text(0.03, 0.003,
        "*** p<0,001   ** p<0,01   * p<0,05  |  "
        "OR = Odds Ratio; CI = 95% Confidence Interval  |  "
        "Filled diamond = p < 0.05 | — = Non-estimable OR",
        color=SUBTEXT, fontsize=15, style="italic", fontfamily="Arial")

plt.tight_layout(rect=[0, 0.018, 1, 0.970])
plt.savefig(OUTPUT_PNG, dpi=180, bbox_inches="tight", facecolor=BG)
print(f"\n Gráfico salvo em: {OUTPUT_PNG}")
plt.show()
```